Slotbook — Take-Home Assignment

Nov 4, 2026 · @Rachit

The brief

Build **Slotbook**: hosts publish when they are free, guests book a slot, and nobody is ever double-booked — even when the network, the clock or the user misbehaves.

Think of it as a small Calendly whose booking engine you would trust with money. The core is a full-stack TypeScript app on Postgres. A clean, reliable core matters more than a long feature list, so build the core and its reliability first, then pick from the optional extensions if you have time.

- **Timebox:** the core is sized for about 4–5 hours with an AI coding assistant. Optional extensions are on top of that.
- **AI tools:** Claude, Cursor, Copilot or anything else — use them as much as you like.
- **Deadline:** submit within 72 hours of receiving this brief.
- **Unfinished work:** fine. Write down what is missing and why in your report.

Core features

Three surfaces: a host dashboard, a public booking page per event type, and a JSON API that both use.

Host (authenticated — a seeded login or magic link is fine)

1. Set a home time zone (IANA, e.g. `Asia/Kolkata`, `America/New_York`).
2. Create event types: name, slug, duration (15–120 min), capacity (1 = one-on-one, more than 1 = group session), buffer before and after (0–30 min), minimum notice (e.g. 4 h) and booking horizon (e.g. 30 days).
3. Define weekly availability in the host's time zone (e.g. Mon–Fri 09:00–17:00, several windows per day allowed) plus date overrides (a day off, or custom hours on one date).
4. See upcoming bookings and cancel one with a reason.

Guest (public, no login)

1. Open `/{host}/{event-slug}` and see available slots for the next 30 days in *their* browser time zone, with a time-zone switcher.
2. Pick a slot — it is **held** for 5 minutes while they enter name and email.

3. Confirm — the booking is **confirmed** and they get a private manage link to view, reschedule or cancel.
4. Reschedule moves the booking to a new slot.

Booking lifecycle

A booking moves `held` → `confirmed` → optionally `cancelled`. A hold that is not confirmed in time becomes `expired`. Capacity counts holds and confirmed bookings, never expired or cancelled ones.

Notifications

On confirm, reschedule and cancel, emit an event to a notifications sink. A real email provider is not needed — writing to a table or posting to a local webhook receiver is enough.

Minimum API (shape it as you like, but document it)

Method	Path	Purpose
GET	`/api/hosts/{host}/events/{slug}/slots?from&to&tz`	Available slots
POST	`/api/holds`	Hold a slot (takes an `Idempotency-Key` header)
POST	`/api/holds/{id}/confirm`	Confirm with guest details
POST	`/api/bookings/{token}/reschedule`	Move to a new slot
POST	`/api/bookings/{token}/cancel`	Cancel

Tech constraints and scope

- **Language:** TypeScript end to end.
- **Database:** Postgres 15+ (local Docker or Supabase). Postgres is the source of truth for correctness — no Redis or external lock service.
- **Framework:** your choice — Next.js, Remix, or a Node API (Fastify, Hono, Express) with a React frontend. Any ORM or plain SQL.
- **One-command run:** `docker compose up` (or `pnpm i && pnpm dev` against a compose-provided Postgres) starts the app, runs migrations and seeds a demo host with two event types.
- **UI:** plain and clear is fine; polished visual design is not required.

Out of scope for the core: calendar sync, payments, real email delivery, multi-host round-robin. Some of these appear as optional extensions below.

Optional extensions

Pick any, in any order, once the core and its reliability requirements are in place. Each one should keep every core requirement true.

Extension	What it means
Waitlist	When a slot is full, guests join a waitlist; a cancellation offers the slot to the next person with a time-limited hold
Reminders	Send a reminder 24 h and 1 h before each booking; reschedules and cancellations update or cancel pending reminders
Calendar invite	Attach an `.ics` file to confirmations; updates and cancellations send matching `.ics` changes
Host webhooks	Hosts register a URL and receive signed (HMAC) booking events, with retries and a delivery log
Round-robin team events	One event type served by several hosts; bookings spread across whoever is free
External busy times	Hosts import busy blocks (a mocked calendar feed or an uploaded `.ics`) that remove overlapping slots
Daily cap and per-guest limit	A host takes at most N bookings per local day; a guest holds at most one active booking per event type
Load test	A k6 or Artillery script against the booking path, with results (throughput, p95 latency, error rate) in your report
End-to-end tests and CI	Playwright tests for the guest flow; a GitHub Actions workflow running all tests on every push
Observability	Structured logs with request IDs, a `/health` endpoint and basic metrics (bookings, hold expiries, notification lag)

Extension	What it means
Abuse protection	Rate limits on public endpoints, plus protection against bots holding every slot
Live deployment	A public URL (e.g. Vercel + Supabase, Fly.io, Render) with a seeded demo host
Embeddable widget	A script or iframe snippet that puts the booking page on any website
Accessibility	The guest flow works fully by keyboard and with a screen reader

Submission

Submit two things: a **public GitHub repo** and a **report** (PDF or slide deck) that links to it.

The repo

- [ ] Source, migrations and seed script
- [ ] Tests runnable with one command, including the concurrency test against real Postgres
- [ ] `README.md` — how to run, the API, and a 60-second tour of the UI
- [ ] `AI_LOG.md` — a short note on how you used AI: key prompts, where it got things wrong, what you changed by hand (raw transcripts optional)
- [ ] Full commit history — please don't squash
- [ ] A list of the optional extensions you built

The report (PDF, 6–12 pages, or a deck of 8–15 slides)

1. **Links** — the public GitHub repo, and the live URL if you deployed one
2. **Architecture** — a diagram of the components and how a booking request flows through them
3. **Data model** — the main tables, their key columns and constraints
4. **Key decisions** — the 5–8 choices that matter most, each with the alternatives you considered and why you chose what you did
5. **Reliability** — the failure cases you identified and designed for, how your system handles each, and which test proves it
6. **Time zones** — how slots are generated and displayed, with one daylight-saving example worked through
7. **Optional extensions** — what you built and how each fits the architecture

8. **Testing** — what is covered, how to run it, and results (including load-test numbers if you did one)
9. **Using AI** — how you worked with it, where it helped, where it misled you
10. **Known gaps and next steps** — what is missing, what you would do with another day

Send the repo link and the report to the recruiter who shared this brief.

After you submit, and FAQ

Can I use a starter template or boilerplate? Yes. Say which one in your README.

Can I use Supabase features like RLS or auth? Yes, as long as you explain what they guarantee and what they don't.

Do I have to build optional extensions? No. They are there if you want to go further.

What if I hit the timebox before finishing? Submit what you have and list what is missing in the report.

Something in the brief is unclear. Make a reasonable assumption, note it in the report, and keep going.